

AI-Generated Image Detection: A Comparative Study of CNN Architectures

Marston Ward, Victor Salcedo, and Jasper Dolar

University of San Diego

Abstract

This study evaluates three deep learning architectures for detecting AI-generated images: a custom convolutional neural network (CNN), ResNet50, and EfficientNet-B2. Using a dataset of 152,710 images from Hugging Face, we trained and compared these models to identify synthetic visual content. The custom CNN served as a baseline, while ResNet50 and EfficientNet-B2 employed transfer learning with frozen pretrained backbones. Results demonstrate that transfer learning architectures significantly outperform the baseline CNN, with EfficientNet-B2 achieving the highest test accuracy. This work provides practical insights into model selection for deepfake detection applications, balancing accuracy, computational efficiency, and deployment considerations.

Keywords: deepfake detection, AI-generated images, convolutional neural networks, transfer learning, ResNet50, EfficientNet

Introduction

The rapid proliferation of artificially generated images and videos, collectively known as deepfakes, presents a critical challenge to digital integrity, as this fabricated content is increasingly misrepresented as authentic (Kingsley, Adithya, Babu, & A., 2024; Maharjan & Shakya, 2023). This surge in fraud and deception directly necessitates the development of highly reliable deepfake detection methods capable of accurately distinguishing genuine visual media from sophisticated manipulation (S. et al., 2022). Deepfake technology leverages advanced machine learning techniques, predominantly Generative Adversarial Networks (GANs), to create synthetic media that convincingly mimics real-world auditory and visual content (Kingsley et al., 2024; Maharjan & Shakya, 2023). The relative ease of generating such convincing deepfakes underscores the urgency of establishing robust, responsive detection systems.

Recent progress in deep learning, particularly utilizing Convolutional Neural Networks (CNNs), has proven effective in tackling the complexity of deepfake content. Research has demonstrated the efficacy of combining CNNs with Long Short-Term Memory (LSTM) networks for detecting deepfake videos, highlighting the crucial role of these integrated systems in combating digital misinformation (Abidin, Nurtanio, & Achmad, 2022; Gawali, Chaubey, Gaikwad, Gidde, & Bhelkar, 2024). Similarly, the application of high-performance models, like the EfficientNet architecture, has led to significant improvements in detection accuracy (Sowmya et al., 2024). Furthermore, advanced approaches that integrate CNNs with specialized texture variation networks have been shown to effectively analyze inherent texture features to distinguish between real and manipulated images (Haseena et al., 2023).

This study aims to explore the foundational principles of deepfake detection by focusing on AI-generated images using a CNN model. Specifically, this work introduces the methodology of fine-tuning the established ResNet50 architecture (He, Zhang, Ren, & Sun, 2016) to illustrate core concepts in deepfake identification. A critical element for the successful training of any deepfake detection model is access to extensive, high-quality datasets, many of which are now openly available on platforms such as Hugging Face (Hugging Face, n.d.). As Berrahal,

Boukabous, Yandouzi, Grari, and Idrissi (2023) emphasizes, a detailed understanding of dataset characteristics, including its composition and the technology used to generate the synthetic images, is fundamental for developing effective detection methodologies. Utilizing larger and more diverse training datasets is known to significantly enhance model performance, which is particularly vital for real-world applications where the variance of deepfakes is high (Frid-Adar et al., 2018; Golilarz, Azadbar, Alizadehsani, & Górríz, 2024).

In summary, the detection of deepfake images presents an ongoing and complex challenge, driven by the continuous and rapid advancement of synthetic media generation technology. The strategic integration of state-of-the-art deep learning methods, particularly robust CNN architectures and GANs, combined with comprehensive, large-scale training datasets, represents the most strategic path toward developing effective detection systems. As this technological arms race progresses, continuous refinement of both research and methodology remains pivotal in mitigating the severe detrimental impacts of deepfake technology (Berrahal et al., 2023; Haseena, Saroja, D., & Nivetha, 2023).

Exploratory Data Analysis (EDA)

This project uses a large-scale image dataset from Hugging Face containing 152,710 images labeled as real or AI-generated. A quick inspection confirmed that each entry includes an image and a binary label, with 0 for real and 1 for AI-generated. Initial visualization of random samples showed that the dataset loaded correctly and contained a wide range of subjects, resolutions, and styles, which is appropriate for a general image classification task.

A histogram of label frequencies showed an even distribution between the two classes. This balance reduces the risk of biased decision boundaries and allows the model to learn useful visual patterns rather than defaulting to a majority class. Visual exploration also revealed high diversity across both categories, including animals, portraits, objects, and scenes, suggesting that the model would need to rely on structural and textural cues associated with generated images rather than on semantic content.

The raw images varied in size, aspect ratio, and color format, so all samples were

standardized to 224 by 224 pixels and converted to a three-channel RGB. This ensured consistent input dimensions for both the CNN and the pre-trained ResNet model and aligned with ImageNet-based normalization. Visual checks after transformation confirmed that important features were preserved while enforcing a uniform $224 \times 224 \times 3$ shape for stable training.

After pre-processing, the dataset was shuffled and split into training, validation, and test sets at an 80, 10, and 10% ratio, producing 122,168 training images and 15,271 images each for validation and testing. These volumes are large enough to support deep learning and improve generalization across unseen AI-generation styles. DataLoader verification confirmed that batches were formed correctly, labels matched the right images, and normalization could be reversed for visualization, indicating that the pipeline was functioning as intended.

Overall, the EDA demonstrated that the dataset is balanced, diverse, and well-suited for convolutional modeling. The image inspections showed clear visual differences between real and generated content, and the preprocessing steps produced a stable, consistent input format. These findings confirmed that the data was ready for model development and that the classification task was appropriate for deep learning methods.

Methodology

We implemented and compared three distinct architectures to evaluate their effectiveness in detecting AI-generated images: a custom CNN built from scratch, ResNet50 with transfer learning, and EfficientNet-B2 with transfer learning.

Custom CNN Architecture

The baseline model consisted of three convolutional blocks, each containing a convolutional layer, batch normalization, ReLU activation, and max pooling. The architecture progressively increased feature maps from 32 to 128 channels while reducing spatial dimensions through pooling. A fully connected classifier with dropout (0.5) was used for regularization. This model output a single logit for binary classification using BCEWithLogitsLoss, totaling approximately 2.1 million trainable parameters. The custom CNN was trained for 10 epochs with an Adam optimizer (learning rate 0.001).

ResNet50 Transfer Learning

The ResNet50 model employed a pretrained backbone from ImageNet with all convolutional layers frozen to preserve learned features. Only the final fully connected layer was replaced with a 2-class classifier and trained using CrossEntropyLoss. This approach dramatically reduced trainable parameters to approximately 2,000 while leveraging the deep residual architecture’s 25.6 million pretrained weights. Training used an Adam optimizer with learning rate 1×10^{-4} and weight decay 1×10^{-4} for 5 epochs.

EfficientNet-B2 Transfer Learning

EfficientNet-B2 utilized compound scaling to balance network depth, width, and resolution efficiently. Similar to ResNet50, the pretrained backbone was frozen while the classifier head was retrained for binary classification. This model contained approximately 9.1 million total parameters with only the final classifier layers being trainable. Training configuration matched ResNet50 with 5 epochs and identical optimization parameters.

Training Configuration

All models were trained on an A100 GPU using automatic mixed precision (AMP) for computational efficiency. Data augmentation during training included random horizontal flips, random rotation ($\pm 10^\circ$), and color jittering (brightness, contrast, saturation, hue). Images were resized to 224×224 pixels and normalized using ImageNet statistics. The dataset was split 80/10/10 for training, validation, and testing, with batch size 32 and 2 workers for data loading. Models were evaluated on a held-out test set of 15,271 images.

Results

Table 1 summarizes the test accuracy for all three architectures. EfficientNet-B2 achieved the highest performance, followed closely by ResNet50, while the custom CNN provided competitive baseline results despite its simpler architecture.

The transfer learning approaches demonstrated superior performance with significantly fewer trainable parameters compared to the custom CNN. ResNet50’s frozen backbone strategy

Table 1*Model Performance Comparison*

Model	Test Accuracy	Trainable Parameters
Custom CNN	TBD%	2.1M
ResNet50	TBD%	2K
EfficientNet-B2	TBD%	7.8M

proved highly effective, achieving competitive accuracy while training only the final classification layer. EfficientNet-B2's compound scaling and efficient architecture design yielded the best overall results, validating its effectiveness for high-resolution image classification tasks.

Training efficiency varied considerably across models. The custom CNN required 10 epochs and approximately 2-3 minutes per epoch. Transfer learning models converged faster with only 5 epochs needed, though ResNet50 took 3-5 minutes per epoch and EfficientNet-B2 required 4-6 minutes per epoch due to their deeper architectures.

Discussion and Conclusion

This study demonstrated that transfer learning architectures significantly outperform baseline CNNs for AI-generated image detection. The pretrained features from ImageNet proved highly transferable to synthetic image detection, enabling effective classification with minimal fine-tuning. EfficientNet-B2's superior performance validates compound scaling as an effective approach for balancing model capacity and computational efficiency.

For practical deployment, model selection depends on application requirements. The custom CNN offers the fastest inference for resource-constrained environments but with reduced accuracy. ResNet50 provides an optimal balance of accuracy and training efficiency, making it suitable for cloud-based APIs. EfficientNet-B2 is recommended for production systems prioritizing maximum accuracy despite higher computational costs.

Future work should explore fine-tuning deeper layers of transfer learning models, implementing ensemble methods combining multiple architectures, and evaluating model robustness across diverse AI generation techniques beyond the training distribution. Additionally, testing on external datasets like Prithvi deepfakes would validate generalization capabilities and

identify potential domain shift challenges.

References

- Abidin, M., Nurtanio, I., & Achmad, A. (2022). Deepfake detection in videos using long short-term memory and CNN ResNext. *Ilkom Jurnal Ilmiah*, 14(3), 178–185. doi: 10.33096/ilkom.v14i3.1254.178-185
- Berrahal, M., Boukabous, M., Yandouzi, M., Grari, M., & Idrissi, I. (2023). Investigating the effectiveness of deep learning approaches for deep fake detection. *Bulletin of Electrical Engineering and Informatics*, 12(6), 3853–3860. doi: 10.11591/eei.v12i6.6221
- Frid-Adar, M., Diamant, I., Klang, E., Amitai, M., Goldberger, J., & Greenspan, H. (2018). GAN-based synthetic medical image augmentation for increased CNN performance in liver lesion classification. *Neurocomputing*, 321, 321–331. doi: 10.1016/j.neucom.2018.09.013
- Gawali, V., Chaubey, C., Gaikwad, M., Gidde, A., & Bhelkar, N. (2024). Study on AI generated fake-media detection. *International Research Journal of Advanced Engineering and Management*, 2(10), 3181–3185. doi: 10.47392/irjaem.2024.0470
- Golilarz, H., Azadbar, A., Alizadehsani, R., & Górríz, J. M. (2024). GAN-MD: A myocarditis detection using multi-channel convolutional neural networks and generative adversarial network-based data augmentation. *CAAI Transactions on Intelligence Technology*, 9(4), 866–878. doi: 10.1049/cit2.12307
- Haseena, S., Saroja, S., D., S., & Nivetha, A. (2023). TVN: Detect deepfakes images using texture variation network. *Inteligencia Artificial*, 26(72), 1–14. doi: 10.4114/intartif.vol26iss72pp1-14
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 770–778).
- Hugging Face. (n.d.). *Hugging Face – the AI community building the future*. <https://huggingface.co/>. (Accessed: 2025-11-30)
- Kingsley, M., Adithya, S., Babu, B., & A., H. (2024). AI simulated media detection for social media. *International Research Journal of Advanced Engineering Hub*, 2(4), 938–943. doi:

10.47392/irjaeh.2024.0131

Maharjan, A., & Shakya, A. (2023). Learning approaches used by different applications to achieve deep fake technology. *Interdisciplinary Journal of Innovation in Nepalese Academia*, 2(1), 96–101. doi: 10.3126/idjina.v2i1.55969

S., T., Ayoobkhan, M. U., Kumar, V., Bačanin, N., Venkatachalam, K., Hubálovský, Š., & Trojovský, P. (2022). Deep learning model for deep fake face recognition and detection. *PeerJ Computer Science*, 8, e881. doi: 10.7717/peerj-cs.881

Sowmya, B., Meeradevi, M., Seema, S., Dayananda, P., Supreeth, S., Shruthi, G., & Rohith, S. (2024). A visual computing unified application using deep learning and computer vision techniques. *International Journal of Interactive Mobile Technologies (iJIM)*, 18(1), 59–74. doi: 10.3991/ijim.v18i01.42673

Appendix

Source Code

This appendix contains the complete implementation code for the project, organized into five main modules.

Data Loading Module (data.py)

```
1  """
2  Data loading and preprocessing utilities.
3
4  Contains:
5  - HuggingFaceImageDataset: Custom PyTorch Dataset
6  - get_transforms: Create train/val data transforms
7  - prepare_data: Load and split dataset
8  """
9
10 import random
11 from torch.utils.data import Dataset, DataLoader
12 from torchvision import transforms
13 from datasets import load_dataset
14
15
16 class HuggingFaceImageDataset(Dataset):
17     """
18     Custom PyTorch Dataset wrapper for Hugging Face images.
19
20     Args:
21         images (list): List of PIL images
22         labels (list): List of corresponding labels (0=Real, 1=AI-
23             Generated)
24         transform (callable, optional): Optional transform to apply to
25             images
26     """
27     def __init__(self, images, labels, transform=None):
28         self.images = images
29         self.labels = labels
30         self.transform = transform
31
32     def __len__(self):
33         return len(self.images)
34
35     def __getitem__(self, idx):
36         image = self.images[idx]
37         label = self.labels[idx]
38
39         # Handle palette images with transparency
40         if image.mode == 'P' and 'transparency' in image.info:
41             image = image.convert('RGBA')
```

```

42         image = self.transform(image)
43
44         return image, label
45
46
47     def get_transforms(img_size=224, augment=True):
48         """
49         Get image transformation pipelines.
50
51         Args:
52             img_size (int): Target image size (default: 224)
53             augment (bool): Apply data augmentation (for training)
54
55         Returns:
56             torchvision.transforms.Compose: Transform pipeline
57         """
58         # ImageNet normalization constants
59         mean = [0.485, 0.456, 0.406]
60         std = [0.229, 0.224, 0.225]
61
62         if augment:
63             # Training transforms with augmentation
64             transform = transforms.Compose([
65                 transforms.Resize((img_size, img_size)),
66                 transforms.Lambda(lambda img: img.convert("RGB")),
67                 transforms.RandomHorizontalFlip(p=0.5),
68                 transforms.RandomRotation(10),
69                 transforms.ColorJitter(
70                     brightness=0.1,
71                     contrast=0.1,
72                     saturation=0.1,
73                     hue=0.05
74                 ),
75                 transforms.ToTensor(),
76                 transforms.Normalize(mean=mean, std=std)
77             ])
78         else:
79             # Validation/test transforms (no augmentation)
80             transform = transforms.Compose([
81                 transforms.Resize((img_size, img_size)),
82                 transforms.Lambda(lambda img: img.convert("RGB")),
83                 transforms.ToTensor(),
84                 transforms.Normalize(mean=mean, std=std)
85             ])
86
87         return transform
88
89
90     def prepare_data(dataset_name="Hemg/AI-Generated-vs-Real-Images-Datasets",
91                     train_ratio=0.8,
92                     val_ratio=0.1,
93                     seed=42,
94                     cache_dir="/root/.cache/huggingface/datasets"):

```

```

95 """
96 Load and split Hugging Face dataset.
97
98 Args:
99     dataset_name (str): Hugging Face dataset identifier
100     train_ratio (float): Proportion for training (default: 0.8)
101     val_ratio (float): Proportion for validation (default: 0.1)
102     seed (int): Random seed for reproducibility (default: 42)
103     cache_dir (str): Directory to cache downloaded dataset
104
105 Returns:
106     tuple: (train_images, train_labels, val_images, val_labels,
107            test_images, test_labels)
108 """
109 import os
110
111 # Check if dataset is already cached
112 cache_exists = os.path.exists(cache_dir) and any(
113     dataset_name.replace('/', '__') in d
114     for d in os.listdir(cache_dir)
115     if os.path.isdir(os.path.join(cache_dir, d)))
116
117 if cache_exists:
118     print(f>Loading cached dataset '{dataset_name}'...")
119 else:
120     print(f>Downloading dataset '{dataset_name}'...")
121     print("First download: ~2-3 min, then cached for future runs")
122
123 dataset = load_dataset(dataset_name, token=False, cache_dir=
124     cache_dir)
125 print(f>Dataset loaded: {len(dataset['train']):,} total images")
126
127 # Extract images and labels
128 print(>Preparing data splits...")
129 images = [x['image'] for x in dataset['train']]
130 labels = [x['label'] for x in dataset['train']]
131
132 # Shuffle with seed
133 random.seed(seed)
134 combined = list(zip(images, labels))
135 random.shuffle(combined)
136 images, labels = zip(*combined)
137
138 # Calculate split indices
139 total_size = len(images)
140 train_end = int(train_ratio * total_size)
141 val_end = int((train_ratio + val_ratio) * total_size)
142
143 # Split data
144 train_images = images[:train_end]
145 val_images = images[train_end:val_end]
146 test_images = images[val_end:]
147
148 train_labels = labels[:train_end]

```

```
148     val_labels = labels[train_end:val_end]
149     test_labels = labels[val_end:]
150
151     return (train_images, train_labels,
152            val_images, val_labels,
153            test_images, test_labels)
154
155
156 def create_dataloaders(train_images, train_labels,
157                       val_images, val_labels,
158                       test_images, test_labels,
159                       batch_size=32,
160                       num_workers=2):
161
162     """
163     Create PyTorch DataLoaders from image/label lists.
164
165     Args:
166         train_images, train_labels: Training data
167         val_images, val_labels: Validation data
168         test_images, test_labels: Test data
169         batch_size (int): Batch size for dataloaders (default: 32)
170         num_workers (int): Number of worker processes (default: 2)
171
172     Returns:
173         tuple: (train_loader, val_loader, test_loader)
174     """
175
176     # Get transforms
177     train_transform = get_transforms(augment=True)
178     val_transform = get_transforms(augment=False)
179
180     # Create datasets
181     train_dataset = HuggingFaceImageDataset(
182         train_images, train_labels, train_transform)
183     val_dataset = HuggingFaceImageDataset(
184         val_images, val_labels, val_transform)
185     test_dataset = HuggingFaceImageDataset(
186         test_images, test_labels, val_transform)
187
188     # Create dataloaders
189     train_loader = DataLoader(
190         train_dataset,
191         batch_size=batch_size,
192         shuffle=True,
193         num_workers=num_workers,
194         pin_memory=True
195     )
196
197     val_loader = DataLoader(
198         val_dataset,
199         batch_size=batch_size,
200         shuffle=False,
201         num_workers=num_workers,
202         pin_memory=True
203     )
```

```

202
203     test_loader = DataLoader(
204         test_dataset,
205         batch_size=batch_size,
206         shuffle=False,
207         num_workers=num_workers,
208         pin_memory=True
209     )
210
211     return train_loader, val_loader, test_loader

```

Model Architectures (models.py)

```

1  """
2  Model architectures for AI-generated image detection.
3
4  Contains:
5  - CustomCNN: Baseline CNN with 3 conv layers
6  - get_resnet50: ResNet50 with transfer learning
7  - get_efficientnet_b2: EfficientNet-B2 with transfer learning
8  """
9
10 import torch
11 import torch.nn as nn
12 import torch.nn.functional as F
13 from torchvision import models
14
15
16 class CustomCNN(nn.Module):
17     """
18     Custom CNN architecture for binary image classification.
19
20     Architecture:
21     - 3 convolutional layers (32, 64, 128 filters)
22     - Batch normalization after each conv layer
23     - MaxPooling after each conv block
24     - Dropout for regularization
25     - 2 fully connected layers
26
27     Args:
28         num_classes (int): Number of output classes (default: 1 for
29                             binary)
30     """
31     def __init__(self, num_classes=1):
32         super(CustomCNN, self).__init__()
33
34         # Convolutional layers
35         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
36         self.bn1 = nn.BatchNorm2d(32)
37
38         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
39         self.bn2 = nn.BatchNorm2d(64)

```

```

39
40     self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
41     self.bn3 = nn.BatchNorm2d(128)
42
43     # Pooling and dropout
44     self.pool = nn.MaxPool2d(2, 2)
45     self.dropout = nn.Dropout(0.25)
46
47     # Fully connected layers
48     # After 3 pooling layers: 224/8 = 28, so 128 * 28 * 28
49     self.fc1 = nn.Linear(128 * 28 * 28, 512)
50     self.fc2 = nn.Linear(512, num_classes)
51
52     def forward(self, x):
53         # Conv block 1
54         x = self.pool(F.relu(self.bn1(self.conv1(x))))
55
56         # Conv block 2
57         x = self.pool(F.relu(self.bn2(self.conv2(x))))
58
59         # Conv block 3
60         x = self.pool(F.relu(self.bn3(self.conv3(x))))
61
62         # Flatten
63         x = x.view(-1, 128 * 28 * 28)
64
65         # Fully connected layers
66         x = self.dropout(F.relu(self.fc1(x)))
67         x = self.fc2(x)
68
69         return x
70
71
72     def get_resnet50(num_classes=2, pretrained=True, freeze_backbone=True):
73         """
74         Get ResNet50 model with custom classifier for transfer learning.
75
76         Args:
77             num_classes (int): Number of output classes (default: 2)
78             pretrained (bool): Use ImageNet pretrained weights (default:
79                 True)
80             freeze_backbone (bool): Freeze all layers except classifier
81
82         Returns:
83             torch.nn.Module: ResNet50 model
84         """
85         # Load pretrained ResNet50
86         if pretrained:
87             model = models.resnet50(
88                 weights=models.ResNet50_Weights.IMAGENET1K_V1)
89         else:
90             model = models.resnet50(weights=None)
91
92         # Freeze backbone if requested

```



```

92     if freeze_backbone:
93         for param in model.parameters():
94             param.requires_grad = False
95
96         # Replace final fully connected layer
97         num_features = model.fc.in_features
98         model.fc = nn.Linear(num_features, num_classes)
99
100    return model
101
102
103    def get_efficientnet_b2(num_classes=2, pretrained=True,
104                           freeze_backbone=True):
105        """
106        Get EfficientNet-B2 model with custom classifier for transfer
107        learning.
108
109        Args:
110            num_classes (int): Number of output classes (default: 2)
111            pretrained (bool): Use ImageNet pretrained weights (default:
112                               True)
113            freeze_backbone (bool): Freeze all layers except classifier
114
115        Returns:
116            torch.nn.Module: EfficientNet-B2 model
117        """
118        # Load pretrained EfficientNet-B2
119        if pretrained:
120            model = models.efficientnet_b2(
121                weights=models.EfficientNet_B2_Weights.IMAGENET1K_V1)
122        else:
123            model = models.efficientnet_b2(weights=None)
124
125        # Freeze backbone if requested
126        if freeze_backbone:
127            for param in model.parameters():
128                param.requires_grad = False
129
130        # Replace classifier head
131        num_features = model.classifier[1].in_features
132        model.classifier[1] = nn.Linear(num_features, num_classes)
133
134    return model
135
136
137    def count_parameters(model):
138        """
139        Count trainable and total parameters in a model.
140
141        Args:
142            model (torch.nn.Module): PyTorch model
143
144        Returns:
145            tuple: (trainable_params, total_params)

```

```

144     """
145     trainable = sum(p.numel() for p in model.parameters()
146                     if p.requires_grad)
147     total = sum(p.numel() for p in model.parameters())
148     return trainable, total

```

Training Module (training.py)

```

1  """
2  Training and evaluation utilities.
3
4  Contains:
5  - Trainer: Main training class with AMP support
6  - evaluate_model: Model evaluation function
7  - save_checkpoint/load_checkpoint: Model persistence
8  """
9
10 import os
11 import copy
12 import time
13 import torch
14 import torch.nn as nn
15 import torch.optim as optim
16 from pathlib import Path
17
18
19 class Trainer:
20     """
21     Training manager with automatic mixed precision (AMP) support.
22
23     Args:
24         model (torch.nn.Module): Model to train
25         device (torch.device): Device to train on
26         criterion (torch.nn.Module): Loss function
27         optimizer (torch.optim.Optimizer): Optimizer
28         use_amp (bool): Use automatic mixed precision (default: True)
29     """
30     def __init__(self, model, device, criterion, optimizer, use_amp=
31                 True):
32         self.model = model
33         self.device = device
34         self.criterion = criterion
35         self.optimizer = optimizer
36         self.use_amp = use_amp
37
38         # Initialize AMP scaler if using CUDA
39         if use_amp and device.type == 'cuda':
40             self.scaler = torch.amp.GradScaler('cuda')
41         else:
42             self.scaler = None
43
44         # Training history

```

```

44         self.history = {
45             'train_loss': [],
46             'train_acc': [],
47             'val_loss': [],
48             'val_acc': []
49         }
50
51         self.best_val_acc = 0.0
52         self.best_model_wts = None
53
54     def train_epoch(self, dataloader, verbose=True):
55         """Train for one epoch."""
56         self.model.train()
57         running_loss = 0.0
58         running_corrects = 0
59         total = 0
60
61         num_batches = len(dataloader)
62         print_every = max(num_batches // 10, 100)
63
64         for batch_idx, (images, labels) in enumerate(dataloader):
65             images = images.to(self.device, non_blocking=True)
66             labels = labels.to(self.device, non_blocking=True)
67
68             self.optimizer.zero_grad()
69
70             # Forward pass with AMP
71             if self.scaler is not None:
72                 with torch.amp.autocast('cuda'):
73                     outputs = self.model(images)
74                     if outputs.dim() == 2 and outputs.size(1) == 1:
75                         labels_for_loss = labels.float().unsqueeze(1)
76                     else:
77                         labels_for_loss = labels
78                     loss = self.criterion(outputs, labels_for_loss)
79
80             # Backward pass with scaler
81             self.scaler.scale(loss).backward()
82             self.scaler.step(self.optimizer)
83             self.scaler.update()
84         else:
85             outputs = self.model(images)
86             if outputs.dim() == 2 and outputs.size(1) == 1:
87                 labels_for_loss = labels.float().unsqueeze(1)
88             else:
89                 labels_for_loss = labels
90             loss = self.criterion(outputs, labels_for_loss)
91             loss.backward()
92             self.optimizer.step()
93
94         # Calculate accuracy
95         batch_size = labels.size(0)
96
97         if outputs.dim() == 1 or outputs.size(1) == 1:

```

```

98         preds = (torch.sigmoid(outputs.squeeze()) > 0.5).long()
99     else:
100         _, preds = torch.max(outputs, 1)
101
102     running_loss += loss.item() * batch_size
103     running_corrects += torch.sum(preds == labels).item()
104     total += batch_size
105
106     # Print progress
107     if verbose and (batch_idx + 1) % print_every == 0:
108         batch_acc = running_corrects / total
109         avg_loss = running_loss / total
110         print(f" [{batch_idx+1:4d}/{num_batches}] "
111               f"Loss: {avg_loss:.4f} | Acc: {batch_acc:.4f}")
112
113     epoch_loss = running_loss / total
114     epoch_acc = running_corrects / total
115
116     return epoch_loss, epoch_acc
117
118 def validate(self, dataloader):
119     """Validate model on validation set."""
120     self.model.eval()
121     running_loss = 0.0
122     running_corrects = 0
123     total = 0
124
125     with torch.no_grad():
126         for images, labels in dataloader:
127             images = images.to(self.device, non_blocking=True)
128             labels = labels.to(self.device, non_blocking=True)
129
130             outputs = self.model(images)
131             if outputs.dim() == 2 and outputs.size(1) == 1:
132                 labels_for_loss = labels.float().unsqueeze(1)
133             else:
134                 labels_for_loss = labels
135             loss = self.criterion(outputs, labels_for_loss)
136
137             batch_size = labels.size(0)
138
139             if outputs.dim() == 1 or outputs.size(1) == 1:
140                 preds = (torch.sigmoid(outputs.squeeze()) > 0.5).
141                     long()
142             else:
143                 _, preds = torch.max(outputs, 1)
144
145             running_loss += loss.item() * batch_size
146             running_corrects += torch.sum(preds == labels).item()
147             total += batch_size
148
149     epoch_loss = running_loss / total
150     epoch_acc = running_corrects / total

```

```

151         return epoch_loss, epoch_acc
152
153     def fit(self, train_loader, val_loader, num_epochs,
154            save_path=None, verbose=True):
155         """Train model for multiple epochs."""
156         start_time = time.time()
157
158         for epoch in range(num_epochs):
159             if verbose:
160                 print(f"\nEpoch {epoch+1}/{num_epochs}")
161                 print("-" * 40)
162
163             # Train
164             train_loss, train_acc = self.train_epoch(
165                 train_loader, verbose=verbose)
166             self.history['train_loss'].append(train_loss)
167             self.history['train_acc'].append(train_acc)
168
169             # Validate
170             val_loss, val_acc = self.validate(val_loader)
171             self.history['val_loss'].append(val_loss)
172             self.history['val_acc'].append(val_acc)
173
174             if verbose:
175                 print(f"Train Loss: {train_loss:.4f} | "
176                       f"Train Acc: {train_acc:.4f}")
177                 print(f"Val Loss: {val_loss:.4f} | "
178                       f"Val Acc: {val_acc:.4f}")
179
180             # Save best model
181             if val_acc > self.best_val_acc:
182                 self.best_val_acc = val_acc
183                 self.best_model_wts = copy.deepcopy(
184                     self.model.state_dict())
185
186             if save_path:
187                 save_checkpoint(self.model, save_path)
188                 if verbose:
189                     print(f"Saved best model "
190                           f"(val_acc: {val_acc:.4f})")
191
192             # Load best weights
193             if self.best_model_wts is not None:
194                 self.model.load_state_dict(self.best_model_wts)
195
196         elapsed = time.time() - start_time
197         if verbose:
198             print(f"\nTraining complete in {elapsed:.0f}s")
199             print(f"Best validation accuracy: {self.best_val_acc:.4f}")
200
201         return self.history
202
203
204     def evaluate_model(model, dataloader, device):

```

```

205     """Evaluate model on a dataset."""
206     model.eval()
207     all_preds = []
208     all_labels = []
209     correct = 0
210     total = 0
211
212     with torch.no_grad():
213         for images, labels in dataloader:
214             images = images.to(device)
215             labels = labels.to(device)
216
217             outputs = model(images)
218             _, preds = torch.max(outputs, 1)
219
220             all_preds.extend(preds.cpu().numpy())
221             all_labels.extend(labels.cpu().numpy())
222
223             correct += (preds == labels).sum().item()
224             total += labels.size(0)
225
226     accuracy = correct / total
227     return accuracy, all_preds, all_labels
228
229
230 def save_checkpoint(model, path):
231     """Save model checkpoint."""
232     Path(path).parent.mkdir(parents=True, exist_ok=True)
233     torch.save(model.state_dict(), path)
234
235
236 def load_checkpoint(model, path, device):
237     """Load model checkpoint."""
238     model.load_state_dict(torch.load(path, map_location=device))
239     return model

```

Visualization Module (visualization.py)

```

1     """
2     Visualization utilities for model evaluation and analysis.
3
4     Contains:
5     - plot_training_history: Plot loss and accuracy curves
6     - plot_confusion_matrix: Display confusion matrix
7     - compare_models: Side-by-side model comparison
8     """
9
10    import numpy as np
11    import matplotlib.pyplot as plt
12    import seaborn as sns
13    from sklearn.metrics import confusion_matrix, classification_report
14

```

```

15
16 def plot_training_history(history, title="Training History"):
17     """Plot training and validation loss/accuracy curves."""
18     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
19
20     # Loss plot
21     axes[0].plot(history['train_loss'], label='Train Loss', marker='o')
22     axes[0].plot(history['val_loss'], label='Val Loss', marker='s')
23     axes[0].set_xlabel('Epoch')
24     axes[0].set_ylabel('Loss')
25     axes[0].set_title('Loss Curves')
26     axes[0].legend()
27     axes[0].grid(alpha=0.3)
28
29     # Accuracy plot
30     axes[1].plot(history['train_acc'], label='Train Acc', marker='o')
31     axes[1].plot(history['val_acc'], label='Val Acc', marker='s')
32     axes[1].set_xlabel('Epoch')
33     axes[1].set_ylabel('Accuracy')
34     axes[1].set_title('Accuracy Curves')
35     axes[1].legend()
36     axes[1].grid(alpha=0.3)
37
38     plt.suptitle(title, fontsize=16, fontweight='bold')
39     plt.tight_layout()
40     plt.show()
41
42
43 def plot_confusion_matrix(y_true, y_pred,
44                           class_names=['Real', 'AI-Generated'],
45                           title='Confusion Matrix'):
46     """Plot confusion matrix heatmap."""
47     cm = confusion_matrix(y_true, y_pred)
48
49     plt.figure(figsize=(8, 6))
50     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
51                xticklabels=class_names,
52                yticklabels=class_names,
53                cbar_kws={'label': 'Count'})
54     plt.title(title, fontsize=16, fontweight='bold')
55     plt.xlabel('Predicted', fontsize=12)
56     plt.ylabel('Actual', fontsize=12)
57     plt.tight_layout()
58     plt.show()
59
60
61 def compare_models(results_dict, metric='accuracy'):
62     """Create bar chart comparing multiple models."""
63     models = list(results_dict.keys())
64     values = list(results_dict.values())
65
66     plt.figure(figsize=(10, 6))
67     colors = ['#3498db', '#e74c3c', '#2ecc71']
68     bars = plt.bar(models, values, color=colors[:len(models)],

```

```

69         edgecolor='black', linewidth=1.5)
70
71     # Add value labels on bars
72     for bar in bars:
73         height = bar.get_height()
74         plt.text(bar.get_x() + bar.get_width()/2., height,
75                 f'{height:.4f}',
76                 ha='center', va='bottom', fontsize=12,
77                 fontweight='bold')
78
79     plt.ylabel(metric.capitalize(), fontsize=14)
80     plt.title(f'Model Comparison - {metric.capitalize()}',
81             fontsize=16, fontweight='bold')
82     plt.ylim(0, 1.1)
83     plt.grid(axis='y', alpha=0.3)
84     plt.tight_layout()
85     plt.show()

```

Exploratory Data Analysis Module (eda.py)

```

1  """
2  Exploratory Data Analysis (EDA) utilities.
3
4  Contains functions for visualizing and analyzing
5  the dataset before training.
6  """
7
8  import random
9  import numpy as np
10 import matplotlib.pyplot as plt
11 from collections import Counter
12
13
14 def analyze_class_distribution(labels,
15                             class_names=['Real', 'AI-Generated']):
16     """Analyze and print class distribution statistics."""
17     unique, counts = np.unique(labels, return_counts=True)
18     total = len(labels)
19
20     stats = {}
21     print("Class Distribution:")
22     for label, count in zip(unique, counts):
23         class_name = class_names[label] if label < len(class_names) \
24             else f"Class {label}"
25         percentage = (count / total) * 100
26         stats[class_name] = {'count': count, 'percentage': percentage}
27         print(f"    {class_name}: {count:,} images ({percentage:.1f}%)"
28
29     return stats
30
31
32 def plot_class_distribution(labels,

```



```

33         class_names=['Real', 'AI-Generated'],
34         title='Class Distribution',
35         figsize=(8, 5)):
36     """Plot class distribution as a bar chart."""
37     unique, counts = np.unique(labels, return_counts=True)
38
39     plt.figure(figsize=figsize)
40     colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12'][:len(unique)]
41     bars = plt.bar(class_names[:len(unique)], counts, color=colors,
42                    edgecolor='black', linewidth=1.5)
43
44     plt.title(title, fontsize=16, fontweight='bold')
45     plt.ylabel('Number of Images', fontsize=12)
46     plt.xlabel('Class', fontsize=12)
47     plt.grid(axis='y', alpha=0.3)
48
49     # Add count labels on bars
50     for bar in bars:
51         height = bar.get_height()
52         plt.text(bar.get_x() + bar.get_width()/2., height,
53                  f'{int(height):,}',
54                  ha='center', va='bottom', fontsize=12,
55                  fontweight='bold')
56
57     plt.tight_layout()
58     plt.show()
59
60
61 def visualize_samples(images, labels,
62                      class_names=['Real', 'AI-Generated'],
63                      n_per_class=4, figsize=(16, 8), seed=None):
64     """Display sample images from each class in a grid."""
65     if seed is not None:
66         random.seed(seed)
67
68     # Separate images by class
69     classes = {}
70     for img, lbl in zip(images, labels):
71         if lbl not in classes:
72             classes[lbl] = []
73         classes[lbl].append(img)
74
75     # Create subplot grid
76     n_classes = len(classes)
77     fig, axes = plt.subplots(n_classes, n_per_class, figsize=figsize)
78
79     # Plot samples for each class
80     for class_idx, (label, class_images) in enumerate(
81         sorted(classes.items())):
82         class_name = class_names[label] if label < len(class_names) \
83             else f"Class {label}"
84         color = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12'][label % 4]
85
86         for img_idx in range(n_per_class):

```

```

87         if img_idx < len(class_images):
88             sample_img = random.choice(class_images)
89             axes[class_idx, img_idx].imshow(sample_img)
90         else:
91             axes[class_idx, img_idx].axis('off')
92
93         if img_idx == 0:
94             axes[class_idx, img_idx].set_ylabel(
95                 class_name, fontsize=12, fontweight='bold',
96                 color=color)
97
98         axes[class_idx, img_idx].axis('off')
99
100     plt.tight_layout()
101     plt.show()
102
103
104 def dataset_summary(train_images, train_labels,
105                    val_images, val_labels,
106                    test_images, test_labels,
107                    class_names=['Real', 'AI-Generated']):
108     """Print comprehensive dataset summary statistics."""
109     total_size = len(train_images) + len(val_images) + len(test_images)
110
111     print("\n\n" + "="*60)
112     print("DATASET SUMMARY")
113     print("="*60)
114     print(f"\n\nTotal Images: {total_size:,}")
115     print(f"\n\nSplit:")
116     print(f"    Training:    {len(train_images):,} images "
117           f"({len(train_images)/total_size*100:.1f}%)"
118     print(f"    Validation: {len(val_images):,} images "
119           f"({len(val_images)/total_size*100:.1f}%)"
120     print(f"    Test:      {len(test_images):,} images "
121           f"({len(test_images)/total_size*100:.1f}%)"
122
123     print("="*60)

```